PLAN FORMATIVO BACKEND JAVA

CASO PRÁCTICO – MÓDULO 2

Arquitectura por Capas y Persistencia con PostgreSQL

Duración estimada: 16 horas

1. Contexto del módulo

En el módulo anterior se construyó una primera versión del sistema NovaBank como aplicación de consola en Java. El sistema permitía gestionar clientes, cuentas, operaciones financieras y consultas, con todos los datos almacenados en memoria.

En este módulo el sistema da un salto arquitectónico importante: se introduce persistencia real con PostgreSQL y se refactoriza el código para adoptar una arquitectura por capas bien definida. Al finalizar este módulo, NovaBank será una aplicación que persiste sus datos en una base de datos relacional y cuyo código está organizado de forma profesional.

> **Nota**
>
> ¿Qué es refactorizar?
> Refactorizar significa reorganizar y mejorar el código existente sin cambiar su comportamiento externo. El sistema debe seguir haciendo exactamente lo mismo que en el Módulo 1, pero con una estructura interna más limpia, mantenible y preparada para crecer.
> La refactorización es una habilidad fundamental en el desarrollo profesional. Los sistemas reales se refactorizan continuamente a medida que crecen y evolucionan.

1.1 Objetivos del módulo

Al finalizar este módulo el alumno habrá trabajado los siguientes objetivos:

- Refactorizar el sistema para adoptar una arquitectura por capas.
- Sustituir el almacenamiento en memoria por persistencia real con PostgreSQL.
- Aplicar el patrón Repository para aislar el acceso a datos.
- Introducir y comprender los patrones de diseño: Singleton, Factory, Builder, Strategy y Decorator.
- Aplicar programación funcional con Streams y Lambdas de Java.
- Gestionar transacciones JDBC para garantizar la atomicidad de operaciones críticas.
- Mantener y ampliar la suite de tests del módulo anterior.

> **Nota**
>
> Sobre los patrones de diseño en este módulo.
> Este módulo introduce cinco patrones clásicos: Singleton, Factory, Builder, Strategy y Decorator. Todos forman parte del alcance del módulo y el alumno debe comprenderlos y saber identificar cuándo aplicarlos, de modo que sean implementados sólo donde aporten valor real sin forzar su uso.

2. Arquitectura por capas: el resultado esperado

El salto más importante de este módulo no es técnico: es conceptual. El sistema ya funciona, pero toda su lógica convive sin una separación clara de responsabilidades. En este módulo se refactoriza el código para adoptar una arquitectura en capas.

Esta sección describe el resultado esperado: qué capas debe tener el sistema, qué responsabilidad tiene cada una, qué principios deben guiar las decisiones de diseño y cómo reconocer cuando una decisión está bien tomada. La estructura concreta de paquetes y clases es una decisión de diseño que corresponde al alumno, no algo que se prescribe aquí.

2.1 Las cuatro capas del sistema

El objetivo es que el código quede organizado en cuatro capas con responsabilidades bien delimitadas. La regla fundamental es sencilla: las dependencias solo fluyen hacia abajo. Cada capa solo conoce a la que tiene inmediatamente por debajo; nunca salta capas ni accede a capas superiores.

```
┌──────────────────────────────────────────┐
│          CAPA DE PRESENTACIÓN            │
│  Menús de consola · Lectura de entrada   │
│  Visualización de resultados y errores   │
└──────────────────────────────────────────┘
                      │ llama a
                      ▼
┌──────────────────────────────────────────┐
│           CAPA DE SERVICIOS              │
│  Lógica de negocio · Validaciones        │
│  Coordinación de operaciones complejas   │
└──────────────────────────────────────────┘
                      │ usa
                      ▼
┌──────────────────────────────────────────┐
│            CAPA DE DOMINIO               │
│  Entidades: Cliente, Cuenta, Movimiento  │
└──────────────────────────────────────────┘


┌──────────────────────────────────────────┐
│          CAPA DE PERSISTENCIA            │
│  Repositorios · Consultas SQL · JDBC     │
│  Mapeo de ResultSet a objetos Java       │
└──────────────────────────────────────────┘
                      │
                 PostgreSQL
```

> **Nota**
> El dominio no depende de ninguna otra capa. Es la más estable del sistema: si mañana el sistema pasa de consola a API REST, o de PostgreSQL a MongoDB, las entidades del dominio no deberían cambiar. Por eso en el diagrama aparece en posición central: todas las demás capas lo usan, pero él no depende de nadie.

2.2 Responsabilidades de cada capa

Capa de presentación

Es la única capa que interactúa directamente con el usuario. Su responsabilidad es mostrar los menús, leer la entrada por consola y presentar los resultados. No debe contener ninguna lógica de negocio ni acceder directamente a los datos.

Una prueba útil: si mañana el sistema dejara de ser una aplicación de consola y pasara a ser una API REST, ¿qué habría que cambiar? Solo esta capa. Si al eliminar la presentación hubiera que mover lógica de negocio a otro sitio, esa lógica no debería estar aquí.

Capa de servicios

Es el corazón del sistema. Aquí vive toda la lógica de negocio: las validaciones (¿el saldo es suficiente para la retirada?), las reglas del dominio (¿puede este cliente abrir una cuenta?), y la coordinación de operaciones que involucran múltiples entidades — como la transferencia, que toca dos cuentas y genera dos movimientos.

Los servicios no saben nada de cómo se muestran los datos al usuario, ni de cómo se almacenan en la base de datos. Solo coordinan operaciones sobre el dominio, apoyándose en los repositorios para la persistencia.

> **Buena práctica**
>
> Señal de diseño incorrecto: si un servicio tiene imports de clases de consola (Scanner, System.out...) o de SQL (Connection, PreparedStatement...), la separación de responsabilidades está rota. Los servicios solo deberían importar clases del dominio, de otros servicios y de las interfaces de repositorio.

Capa de dominio

Contiene las entidades del sistema: las clases Java que representan los conceptos del negocio bancario (Cliente, Cuenta, Movimiento). Son clases simples — atributos con sus getters y setters — que no contienen lógica de negocio ni acceso a datos. Representan el modelo del mundo real que el sistema gestiona.

Capa de persistencia

Se encarga exclusivamente de hablar con la base de datos. Implementa el patrón Repository: para cada entidad del dominio existe un repositorio responsable de las operaciones de lectura y escritura. El resto del sistema no sabe si los datos vienen de PostgreSQL, de un fichero o de memoria — solo llama a los métodos del repositorio.

El patrón Repository tiene una consecuencia importante para la testabilidad: los servicios pueden probarse de forma completamente aislada sustituyendo los repositorios reales por mocks, sin necesidad de una base de datos activa. Esto se verá en la Sección 11.

> **Buena práctica**
>
> Señal de diseño incorrecto: si en un servicio aparece código SQL directamente, la capa de persistencia no está cumpliendo su función de abstracción. Todo lo que implique hablar con la base de datos debe vivir en los repositorios.

2.3 La regla de dependencias

La regla que no es negociable: las dependencias solo fluyen hacia abajo. Nunca al revés.

```
✓ La presentación llama a los servicios
✓ Los servicios usan los repositorios
```

```
✅ Servicios y repositorios usan las entidades del dominio

❌ Un repositorio llama a un servicio
❌ Un servicio importa clases de presentación (Scanner, System.out...)
❌ La presentación accede directamente al repositorio, saltándose el servicio
❌ Un servicio instancia Connection o PreparedStatement directamente
```

Esta regla es lo que permite reemplazar cualquier capa sin tocar las demás. Si en el Módulo 3 se migra la persistencia de JDBC a Spring Data JPA, los servicios no deberían modificarse en absoluto: solo cambia la implementación de los repositorios. Si los servicios tuviesen SQL hardcodeado, ese cambio implicaría tocar decenas de clases.

2.4 Cómo estructurar el código: paquetes y clases

La arquitectura en capas que se acaba de describir debe reflejarse en la organización física del código. En Java la forma habitual es usar paquetes: cada capa vive en su propio paquete (o familia de subpaquetes) dentro del paquete raíz del proyecto.

No existe una única estructura correcta, pero sí existen principios que guían las buenas decisiones. A continuación se describen los bloques que el sistema necesita y las preguntas de diseño que el alumno tendrá que resolver.

Bloques que el sistema necesita organizar

Independientemente de cómo se nombren los paquetes, el sistema tendrá que ubicar en algún lugar los siguientes bloques:

- Las entidades del dominio (Cliente, Cuenta, Movimiento).
- Los servicios con la lógica de negocio.
- Las interfaces de repositorio — el contrato que usan los servicios.
- Las implementaciones JDBC de esos repositorios.
- El gestor de conexiones a la base de datos.
- Los controladores y vistas de la capa de presentación.
- Los patrones auxiliares: Factory, Builder.
- Utilidades compartidas, si las hay.
- El punto de entrada de la aplicación.

Decisiones de diseño a tomar

Para cada uno de esos bloques hay decisiones que el alumno debe tomar conscientemente y ser capaz de justificar:

Sobre los repositorios: ¿las interfaces van en el mismo paquete que las implementaciones JDBC, o en paquetes separados? Separar la interfaz de la implementación refuerza la idea de que los servicios dependen del contrato, no de la tecnología concreta. Tenerlas juntas es más compacto y suficiente para proyectos pequeños. Ambas opciones son válidas; lo importante es que la decisión sea consciente.

Sobre los patrones auxiliares: ¿Factory y Builder van en paquetes propios, o conviven con la capa a la que más directamente sirven? Un MovimientoFactory está al servicio de la capa de servicios; un ClienteBuilder puede ubicarse junto al dominio. No hay una respuesta única.

Sobre la presentación: en el Módulo 1 probablemente existía una única clase de menú. Ahora que hay controladores y vistas separadas, ¿conviven en un mismo paquete o se separan? ¿Tiene sentido un subpaquete controller y otro view?

Sobre utilidades compartidas: si hay código que no encaja claramente en ninguna capa — generadores de números de cuenta, formateadores, constantes — ¿dónde va? ¿Un paquete util o se coloca cerca de quien lo usa?

Dos enfoques habituales

En proyectos reales se encuentran principalmente dos formas de organizar los paquetes de una aplicación en capas:

Organización por capa técnica: los paquetes reflejan directamente las capas de la arquitectura. Toda la lógica de negocio junta, todos los repositorios juntos, todas las entidades juntas. Es el enfoque más habitual en aplicaciones con un único dominio de negocio como NovaBank.

```
// Organización por capa técnica (habitual en proyectos como NovaBank)
com.novabank
├── model/          ← todas las entidades del dominio
├── service/        ← toda la lógica de negocio
├── repository/     ← interfaces e implementaciones de acceso a datos
├── controller/     ← coordinación entre vista y servicios
├── view/           ← presentación por consola
└── config/         ← configuración (gestor de conexiones, etc.)
```

Organización por funcionalidad de negocio: los paquetes agrupan todo lo relacionado con una funcionalidad concreta — entidad, servicio, repositorio y controlador — en el mismo sitio. Escala mejor cuando el sistema crece y tiene múltiples dominios independientes.

```
// Organización por funcionalidad (más habitual en sistemas grandes)
com.novabank
├── cliente/        ← Cliente.java + ClienteService + ClienteRepository...
├── cuenta/         ← Cuenta.java + CuentaService + CuentaRepository...
├── operacion/      ← lógica de depósitos, retiradas, transferencias
└── config/         ← configuración compartida
```

Para un sistema del tamaño de NovaBank, la organización por capa técnica es la más natural y la más fácil de mantener. Pero cualquiera de las dos es válida si se aplica con consistencia.

Criterios para evaluar la propia estructura

Una vez decidida la organización, estas preguntas sirven para verificar que es buena:

- ¿Puedo describir la responsabilidad de cada paquete en una frase corta y precisa? Si no puedo, el paquete probablemente mezcla responsabilidades.
- ¿Un desarrollador nuevo entendería la estructura sin que nadie se la explique? Si hay que leer el código para entender qué hay en cada paquete, hay margen de mejora.
- ¿Las reglas de dependencia entre capas se respetan? Un import de java.sql.* en un servicio, o de Scanner en un repositorio, es una señal de alerta.

- ¿Hay clases difíciles de ubicar? Si una clase no encaja claramente en ningún paquete, puede ser señal de que hace demasiadas cosas o de que falta un paquete.

Buena práctica

No existe una estructura perfecta, pero sí existen estructuras claramente malas: las que mezclan responsabilidades sin criterio, las que no siguen ningún principio consistente, o las que violan las reglas de dependencia entre capas.

El objetivo es que la estructura sea una decisión de diseño razonada, no el resultado de ir añadiendo clases donde caben.

3. De la memoria a la base de datos: qué cambia

El cambio más significativo de este módulo es la sustitución del almacenamiento en memoria por llamadas reales a PostgreSQL. Pero el impacto de ese cambio en el resto del sistema debería ser mínimo, y entender por qué es una de las lecciones más importantes de este módulo.

En el Módulo 1, los repositorios usaban estructuras Map para guardar los datos. En este módulo, esos mismos repositorios pasan a ejecutar queries SQL contra PostgreSQL. Lo que no cambia — si la arquitectura está bien diseñada — es el contrato que ofrecen al resto del sistema: los servicios siguen llamando a los mismos métodos con los mismos parámetros y esperando los mismos resultados.

> **Nota**
> Esta es la demostración práctica de por qué la capa de persistencia existe: aislar al resto del sistema de los detalles de cómo se almacenan los datos. Si los servicios conocieran los detalles de implementación del repositorio, cada cambio en la tecnología de persistencia implicaría modificar también los servicios.

3.1 El contrato del repositorio: antes y después

La interfaz pública de un repositorio — los métodos que el servicio puede llamar — debería ser idéntica antes y después de la migración. Lo que cambia es únicamente la implementación interna:

Antes (Módulo 1): implementación en memoria

```
// El servicio llama a este método...
public Optional<Cliente> buscarPorId(Long id) {
    return Optional.ofNullable(clientes.get(id)); // busca en un Map
}

public Cliente guardar(Cliente cliente) {
    clientes.put(cliente.getId(), cliente);        // guarda en un Map
    return cliente;
}
```

Ahora (Módulo 2): misma firma, implementación JDBC

```
// ...y sigue llamando al mismo método. No sabe que ahora hay SQL debajo.
public Optional<Cliente> buscarPorId(Long id) {
    String sql = "SELECT * FROM clientes WHERE id = ?";
    try (Connection conn = DatabaseConnectionManager.getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setLong(1, id);
        ResultSet rs = stmt.executeQuery();
        if (rs.next()) {
            return Optional.of(mapearCliente(rs));
        }
        return Optional.empty();
    } catch (SQLException e) {
        throw new RuntimeException("Error al buscar cliente con id " + id, e);
    }
}
```

3.2 El mapeo de ResultSet a objetos Java

Cuando JDBC ejecuta una query, los resultados llegan como un ResultSet: una estructura tabular de filas y columnas, similar a una hoja de cálculo. El trabajo del repositorio es convertir cada fila de ese ResultSet en el objeto Java correspondiente. Este proceso se llama mapeo.

```
// Método de mapeo: convierte una fila del ResultSet en un objeto Cliente
private Cliente mapearCliente(ResultSet rs) throws SQLException {
    return new Cliente(
        rs.getLong("id"),
        rs.getString("nombre"),
        rs.getString("apellidos"),
        rs.getString("dni"),
        rs.getString("email"),
        rs.getString("telefono")
    );
}
```

Este mapeo es manual con JDBC puro: el desarrollador escribe explícitamente qué columna del ResultSet corresponde a qué campo del objeto. En el Módulo 3, Spring Data JPA realizará este proceso de forma automática a partir de anotaciones en la entidad.

3.3 Qué debe cambiar en el proyecto

La migración afecta exclusivamente a la capa de persistencia. El siguiente cuadro resume qué cambia y qué no:

Qué cambia	Qué NO cambia
Las implementaciones de repositorio (pasan de Map a JDBC)	Las interfaces de repositorio (el contrato con los servicios)
Las dependencias Maven (se añade el driver PostgreSQL)	Los servicios: siguen llamando a los mismos métodos
Se añade la clase gestora de conexiones	Las entidades del dominio (Cliente, Cuenta, Movimiento)
Se crea la base de datos y se ejecuta el schema.sql	La capa de presentación y los controladores
Algunos métodos devuelven Optional<T> en lugar de T	Los tests de servicios con Mockito (siguen funcionando igual)

Buena práctica

Si al hacer la migración tienes que modificar los servicios para que funcionen, es una señal de que la separación entre capas no estaba bien hecha en el Módulo 1, o de que la nueva implementación no respeta el contrato de la interfaz. Usa ese momento como oportunidad para identificar y corregir el problema de diseño.

4. Gestión de conexiones a la base de datos

Todas las operaciones JDBC necesitan una Connection activa a PostgreSQL. En lugar de dispersar la lógica de creación de conexiones por cada repositorio, se centralizará en una única clase: DatabaseConnectionManager.

4.1 DatabaseConnectionManager

Esta clase aplica el patrón Singleton para garantizar que exista un único gestor de conexión durante toda la ejecución del programa. Antes de ver el código, es importante entender exactamente qué es lo que el Singleton garantiza aquí:

> **Atención**
>
> Distinción clave: el Singleton del gestor vs. las conexiones individuales.
>
> El patrón Singleton se aplica al objeto DatabaseConnectionManager (el gestor), no a las conexiones individuales a la base de datos.
>
> Cada llamada a getConnection() abre una nueva conexión física a PostgreSQL, lo cual es correcto: JDBC no es thread-safe si se comparte una única conexión entre múltiples operaciones concurrentes.
>
> Lo que se garantiza con Singleton es que haya un único punto centralizado de configuración y obtención de conexiones (URL, usuario, contraseña en un solo lugar). Las conexiones son efímeras y se crean y cierran en cada operación, siempre con try-with-resources.

```
public class DatabaseConnectionManager {

    // Parámetros centralizados (en producción: fichero de propiedades o env vars)
    private static final String URL  = "jdbc:postgresql://localhost:5432/novabank";
    private static final String USER = "postgres";
    private static final String PASS = "tu_password";

    // Singleton: una única instancia del GESTOR en toda la aplicación
    private static DatabaseConnectionManager instance;

    // Constructor privado: nadie puede instanciar esta clase desde fuera
    private DatabaseConnectionManager() {}

    // Único punto de acceso global al gestor
    public static DatabaseConnectionManager getInstance() {
        if (instance == null) {
            instance = new DatabaseConnectionManager();
        }
        return instance;
    }

    // Abre una NUEVA conexión en cada llamada — NO comparte una conexión única
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

> **Nota**
>
> El constructor es private. Esto impide que nadie pueda crear instancias de esta clase desde fuera. La única forma de obtener el gestor es a través de getInstance(). Reflexiona: ¿qué ocurriría si cada repositorio tuviese su propio gestor con credenciales hardcodeadas?

En producción, las credenciales se leen desde variables de entorno o un fichero database.properties en src/main/resources, nunca escritas directamente en el código.

4.2 Uso correcto de try-with-resources

Es fundamental cerrar siempre las conexiones JDBC después de usarlas. La forma más segura es usar try-with-resources, que cierra automáticamente los recursos al salir del bloque, incluso si hay una excepción:

```
// CORRECTO: try-with-resources cierra conn y stmt automáticamente
try (Connection conn = DatabaseConnectionManager.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setLong(1, id);
    ResultSet rs = stmt.executeQuery();
    // ... procesar resultado
} // conn y stmt se cierran aquí, incluso si hay excepción

// INCORRECTO: si hay excepción antes de conn.close(), la conexión queda abierta
Connection conn = DatabaseConnectionManager.getConnection();
// ... operaciones
conn.close(); // puede no ejecutarse si hay una excepción antes → fuga de recursos
```

5. Patrón Repository

El patrón Repository actúa como una capa de abstracción entre la lógica de negocio y el mecanismo de persistencia. El servicio no sabe si los datos vienen de PostgreSQL, de un fichero o de memoria: solo llama a los métodos del repositorio.

Cada repositorio se define como una interfaz y se implementa en una clase JDBC concreta. Además, algunos métodos tienen una variante que acepta una Connection como parámetro; estas variantes son necesarias para participar en transacciones gestionadas desde el servicio, como se explica en la Sección 6.

5.1 ClienteRepository

```
public interface ClienteRepository {
    Cliente guardar(Cliente cliente);
    Optional<Cliente> buscarPorId(Long id);
    Optional<Cliente> buscarPorDni(String dni);
    List<Cliente> listarTodos();
    void eliminar(Long id);
}

// Implementación concreta que accede a PostgreSQL
public class ClienteRepositoryJdbc implements ClienteRepository {
    // ... implementación con try-with-resources y JDBC
}
```

5.2 CuentaRepository

La interfaz define las operaciones estándar y también variantes transaccionales que reciben la conexión activa como parámetro:

```
public interface CuentaRepository {
    Cuenta guardar(Cuenta cuenta);
    Optional<Cuenta> buscarPorId(Long id);
    Optional<Cuenta> buscarPorNumero(String numeroCuenta);
    List<Cuenta> buscarPorClienteId(Long clienteId);
    Cuenta actualizarSaldo(Long cuentaId, BigDecimal nuevoSaldo);

    // Variantes transaccionales: reciben la Connection activa del servicio
    Optional<Cuenta> buscarPorNumero(String numeroCuenta, Connection conn);
    Cuenta actualizarSaldo(Long cuentaId, BigDecimal nuevoSaldo, Connection conn);
}
```

> **Nota**
>
> ¿Por qué dos versiones del mismo método?
>
> Los métodos sin Connection son convenientes para operaciones de lectura simples que no necesitan participar en ninguna transacción: abren y cierran su propia conexión con try-with-resources.
>
> Los métodos que reciben Connection permiten que el servicio controle qué conexión usa el repositorio. De este modo, varias operaciones pueden participar en la misma transacción. Esta distinción se hace visible en el código de la transferencia (Sección 5).

5.3 MovimientoRepository

```
public interface MovimientoRepository {
```

```
    Movimiento guardar(Movimiento movimiento);
    List<Movimiento> buscarPorCuentaId(Long cuentaId);
    List<Movimiento> buscarPorCuentaIdYFechas(
        Long cuentaId,
        LocalDateTime inicio,
        LocalDateTime fin
    );

    // Variante transaccional: recibe la Connection activa
    Movimiento guardar(Movimiento movimiento, Connection conn);
}
```

6. Transacciones JDBC

En sistemas bancarios existen operaciones que deben ejecutarse de forma atómica: o se completan todas las acciones que las componen, o ninguna. El ejemplo más claro es la transferencia entre cuentas.

6.1 El problema de las operaciones parciales

Una transferencia implica al menos tres acciones:

1. Retirar el importe de la cuenta origen.
2. Depositar el importe en la cuenta destino.
3. Registrar los movimientos en la base de datos.

Si el sistema falla después del paso 1 pero antes del paso 2 (por ejemplo, por un corte de red), el dinero ha desaparecido de la cuenta origen pero no ha llegado al destino. Esto deja la base de datos en un estado inconsistente que representa una pérdida real de dinero.

Las transacciones de base de datos garantizan que esto nunca ocurra:

```
-- Si todas las operaciones tienen éxito:
BEGIN;
  UPDATE cuentas SET saldo = saldo - 500 WHERE id = 1;
  UPDATE cuentas SET saldo = saldo + 500 WHERE id = 2;
  INSERT INTO movimientos (...) VALUES (...);
  INSERT INTO movimientos (...) VALUES (...);
COMMIT; -- los cambios se guardan de forma permanente

-- Si algo falla en cualquier punto:
ROLLBACK; -- todos los cambios se deshacen, como si no hubieran ocurrido
```

6.2 Implementación de transacciones en JDBC

Por defecto, JDBC ejecuta cada operación en autocommit: cada sentencia SQL se confirma inmediatamente. Para usar transacciones explícitas hay que desactivar el autocommit y pasar la misma conexión a todos los repositorios que participen:

```
public void transferir(String numeroCuentaOrigen, String numeroCuentaDestino,
                       BigDecimal importe) {
    try (Connection conn = DatabaseConnectionManager.getConnection()) {
        conn.setAutoCommit(false); // desactivar autocommit: inicio de transacción
        try {
            // Paso 1: verificar saldo suficiente
            // Se pasa conn para que use la misma conexión/transacción
            Cuenta origen = cuentaRepository
                .buscarPorNumero(numeroCuentaOrigen, conn)
                .orElseThrow(() ->
                    new CuentaNoEncontradaException(numeroCuentaOrigen));

            if (origen.getSaldo().compareTo(importe) < 0) {
                throw new SaldoInsuficienteException();
            }

            // Paso 2: actualizar saldos — ambos en la misma transacción
            cuentaRepository.actualizarSaldo(
                origen.getId(), origen.getSaldo().subtract(importe), conn);

            Cuenta destino = cuentaRepository
                .buscarPorNumero(numeroCuentaDestino, conn)
                .orElseThrow(() ->
```

```
                    new CuentaNoEncontradaException(numeroCuentaDestino));

            cuentaRepository.actualizarSaldo(
                destino.getId(), destino.getSaldo().add(importe), conn);

            // Paso 3: registrar movimientos — usando MovimientoFactory
            movimientoRepository.guardar(
                MovimientoFactory.crearTransferenciaSaliente(
                    origen.getId(), importe, numeroCuentaDestino), conn);
            movimientoRepository.guardar(
                MovimientoFactory.crearTransferenciaEntrante(
                    destino.getId(), importe, numeroCuentaOrigen), conn);

            conn.commit(); // todo ha ido bien: confirmar
        } catch (Exception e) {
            conn.rollback(); // algo ha fallado: deshacer todos los cambios
            throw new RuntimeException(
                "Error en la transferencia: " + e.getMessage(), e);
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error de conexión", e);
    }
}
```

Atención

Por qué la conexión se pasa como parámetro

Para que los tres pasos de la transferencia participen en la misma transacción JDBC, deben usar exactamente la misma Connection. Por eso las interfaces de repositorio definen variantes que reciben conn como parámetro (Sección 5).

Si cada método del repositorio abriera su propia conexión (nueva llamada a getConnection()), cada operación estaría en una transacción independiente y el ROLLBACK no desharía las operaciones anteriores. Este es precisamente el escenario catastrófico del ejemplo del fallo de red.

7. Patrones de diseño

Este módulo introduce cinco patrones de diseño clásicos aplicados al contexto de NovaBank. Los patrones de diseño son soluciones reutilizables a problemas comunes de diseño de software.

> **Buena práctica**
>
> Criterio de aplicación
>
> No apliques patrones para demostrar que los conoces. Aplícalos porque identificas un problema concreto que resuelven de forma elegante. Un buen desarrollador reconoce cuándo un patrón es la solución adecuada y cuándo añade complejidad innecesaria.
>
> En este módulo: Singleton y Factory son de aplicación directa (obligatorios). Builder se aplica en Cliente y en los tests. Strategy y Decorator son extensiones recomendadas cuya implementación es opcional.

7.1 Singleton

¿Qué es?

El patrón Singleton garantiza que una clase tenga una única instancia durante toda la ejecución del programa, y proporciona un punto de acceso global a ella. Es especialmente útil cuando un componente gestiona un recurso compartido y costoso de inicializar.

Aplicación en NovaBank

DatabaseConnectionManager es el lugar natural para este patrón. Como se explicó en la Sección 4, el Singleton aplica al gestor (la clase que centraliza la configuración), no a las conexiones individuales, que son efímeras y se crean de nuevo en cada operación.

```
public class DatabaseConnectionManager {
    private static DatabaseConnectionManager instance;

    // Constructor privado: impide instanciación desde fuera
    private DatabaseConnectionManager() {}

    // Único punto de acceso global al gestor
    public static DatabaseConnectionManager getInstance() {
        if (instance == null) {
            instance = new DatabaseConnectionManager();
        }
        return instance;
    }

    // Devuelve una NUEVA conexión en cada llamada
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

7.2 Factory

¿Qué es?

El patrón Factory proporciona una forma de delegar la creación de objetos a un método o clase especializada, en lugar de usar new directamente en cada punto del código. Se utiliza cuando la lógica de creación es suficientemente compleja o repetitiva como para centralizarla.

Aplicación en NovaBank

En el sistema NovaBank existen cuatro tipos de movimientos: DEPOSITO, RETIRO, TRANSFERENCIA_ENTRANTE y TRANSFERENCIA_SALIENTE. Sin un Factory, la lógica de construcción de cada tipo de movimiento quedaría dispersa por toda la capa de servicios.

```
public class MovimientoFactory {

    public static Movimiento crearDeposito(Long cuentaId, BigDecimal monto) {
        return new Movimiento(cuentaId, TipoMovimiento.DEPOSITO,
                              monto, "Deposito en cuenta");
    }

    public static Movimiento crearRetiro(Long cuentaId, BigDecimal monto) {
        return new Movimiento(cuentaId, TipoMovimiento.RETIRO,
                              monto, "Retirada de fondos");
    }

    public static Movimiento crearTransferenciaSaliente(
            Long cuentaId, BigDecimal monto, String cuentaDestino) {
        String desc = "Transferencia enviada a " + cuentaDestino;
        return new Movimiento(cuentaId,
            TipoMovimiento.TRANSFERENCIA_SALIENTE, monto, desc);
    }

    public static Movimiento crearTransferenciaEntrante(
            Long cuentaId, BigDecimal monto, String cuentaOrigen) {
        String desc = "Transferencia recibida de " + cuentaOrigen;
        return new Movimiento(cuentaId,
            TipoMovimiento.TRANSFERENCIA_ENTRANTE, monto, desc);
    }
}
```

Buena práctica

La clase de servicio que gestiona una transferencia ya no necesita saber cómo construir un Movimiento. Solo llama a MovimientoFactory.crearTransferenciaSaliente(...). Si en el futuro cambia la forma de construir un movimiento (añadir campos, cambiar el formato de la descripción...), el cambio se hace en un único lugar.

7.3 Builder

¿Qué es?

El patrón Builder permite construir objetos complejos paso a paso, separando la construcción del objeto de su representación final. Es especialmente útil cuando un objeto tiene muchos atributos opcionales o cuando su construcción requiere varios pasos. Una señal clara de que puede ser útil es cuando aparecen constructores con muchos parámetros.

Aplicación en NovaBank

La entidad Cliente tiene varios atributos: nombre, apellidos, dni, email, telefono, fechaCreacion. Un Builder permite construirlo de forma legible y flexible, y resulta especialmente valioso en los tests, donde se necesitan crear objetos de prueba rápidamente con solo los campos relevantes.

```
public class ClienteBuilder {
    private String nombre;
    private String apellidos;
    private String dni;
```

```
    private String email;
    private String telefono;

    public ClienteBuilder nombre(String nombre) {
        this.nombre = nombre; return this;
    }
    public ClienteBuilder apellidos(String apellidos) {
        this.apellidos = apellidos; return this;
    }
    public ClienteBuilder dni(String dni) {
        this.dni = dni; return this;
    }
    public ClienteBuilder email(String email) {
        this.email = email; return this;
    }
    public ClienteBuilder telefono(String telefono) {
        this.telefono = telefono; return this;
    }
    public Cliente build() {
        Cliente c = new Cliente();
        c.setNombre(this.nombre);
        c.setApellidos(this.apellidos);
        c.setDni(this.dni);
        c.setEmail(this.email);
        c.setTelefono(this.telefono);
        return c;
    }
}

// Uso (legible, autoexplicativo):
Cliente cliente = new ClienteBuilder()
    .nombre("Juan")
    .apellidos("Perez Garcia")
    .dni("12345678A")
    .email("juan@email.com")
    .telefono("600123456")
    .build();
```

Buena práctica

Compara con: new Cliente("Juan", "Perez Garcia", "12345678A", "juan@email.com", "600123456").

¿Cuál es más legible? ¿Cuál es más fácil de mantener si el objeto cambia? El Builder también resulta muy útil en los tests unitarios (ver Sección 11).

7.4 Decorator (extensión recomendada)

¿Qué es?

El patrón Decorator permite añadir comportamiento adicional a un objeto de forma dinámica, sin modificar su clase original ni crear subclases. Se basa en envolver el objeto original con otro que extiende su comportamiento. Es una alternativa flexible a la herencia cuando se quieren añadir funcionalidades de forma modular y combinable.

Aplicación en NovaBank

Puede ser útil añadir comportamiento transversal a las operaciones financieras, como registro de trazas (logging) o auditoría, sin modificar el servicio original:

```
// Interfaz común que define el contrato del servicio
```

```
public interface OperacionService {
    void depositar(String numeroCuenta, BigDecimal monto);
    void retirar(String numeroCuenta, BigDecimal monto);
}

// Decorador: añade logging sin tocar OperacionServiceImpl
public class LoggingOperacionService implements OperacionService {
    private final OperacionService delegate; // el servicio real envuelto

    public LoggingOperacionService(OperacionService delegate) {
        this.delegate = delegate;
    }

    @Override
    public void depositar(String numeroCuenta, BigDecimal monto) {
        System.out.println("[LOG] Inicio deposito: cuenta=" + numeroCuenta);
        delegate.depositar(numeroCuenta, monto); // delega en el servicio real
        System.out.println("[LOG] Deposito completado.");
    }

    @Override
    public void retirar(String numeroCuenta, BigDecimal monto) {
        System.out.println("[LOG] Inicio retirada: cuenta=" + numeroCuenta);
        delegate.retirar(numeroCuenta, monto);
        System.out.println("[LOG] Retirada completada.");
    }
}

// Uso: el controlador recibe un OperacionService, no sabe si hay logging o no
OperacionService servicio =
    new LoggingOperacionService(new OperacionServiceImpl(...));
servicio.depositar("ES9121000000000000001", new BigDecimal("500.00"));
```

> **Nota**
> LoggingOperacionService no hereda de OperacionServiceImpl ni modifica su código. Simplemente lo envuelve. En módulos posteriores, cuando trabajes con Spring, verás que este mismo concepto está en la base de los aspectos (AOP): Spring añade comportamiento transversal —logging, transacciones, seguridad— sin que el servicio lo sepa.

7.5 Strategy (extensión recomendada)

¿Qué es?

El patrón Strategy permite definir una familia de algoritmos o comportamientos, encapsular cada uno en una clase separada y hacerlos intercambiables en tiempo de ejecución. Es especialmente útil para evitar grandes bloques if-else o switch que crecen con cada nuevo caso.

Aplicación en NovaBank

En NovaBank existen tres tipos de operaciones financieras con lógicas diferentes. Con Strategy, cada operación se encapsula en su propia clase y el código que las invoca simplemente selecciona la estrategia adecuada:

```
// Interfaz Strategy: contrato común para todas las operaciones
public interface OperacionFinanciera {
    void ejecutar(String numeroCuenta, BigDecimal monto);
}
```

```
// Estrategia concreta: Deposito
public class DepositoStrategy implements OperacionFinanciera {
    private final CuentaRepository cuentaRepository;
    private final MovimientoRepository movimientoRepository;

    public DepositoStrategy(CuentaRepository cuentaRepository,
                            MovimientoRepository movimientoRepository) {
        this.cuentaRepository = cuentaRepository;
        this.movimientoRepository = movimientoRepository;
    }

    @Override
    public void ejecutar(String numeroCuenta, BigDecimal monto) {
        // lógica específica de depósito
    }
}

// Contexto que delega en la estrategia configurada
public class OperacionContext {
    private OperacionFinanciera estrategia;

    public void setEstrategia(OperacionFinanciera estrategia) {
        this.estrategia = estrategia;
    }

    public void ejecutar(String numeroCuenta, BigDecimal monto) {
        estrategia.ejecutar(numeroCuenta, monto);
    }
}

// Uso:
OperacionContext ctx = new OperacionContext();
ctx.setEstrategia(new DepositoStrategy(cuentaRepository, movimientoRepository));
ctx.ejecutar("ES9121000000000000001", new BigDecimal("200.00"));
```

Buena práctica

OperacionContext no sabe si está ejecutando un depósito o un retiro: simplemente delega en la estrategia configurada. Esto facilita añadir nuevas operaciones en el futuro (por ejemplo, una transferencia internacional) sin modificar el código existente.

Este principio se conoce como Open/Closed: abierto a extensión, cerrado a modificación. Es uno de los principios SOLID que verás con mucha frecuencia en proyectos profesionales.

8. Programación funcional con Streams y Lambdas

Java 8 introdujo los Streams y las expresiones lambda, que permiten escribir código más expresivo y conciso al procesar colecciones de datos. En este módulo se introduce su uso en el contexto del sistema NovaBank.

> **Nota**
> Cuándo usar Streams
> Los Streams son especialmente útiles para filtrar, transformar, ordenar y agregar colecciones de datos. Sin embargo, no siempre son la mejor opción: para operaciones simples con una o dos condiciones, un bucle for tradicional puede ser más legible.
> La clave es usar el enfoque más claro para cada situación, no usar Streams porque sí.

8.1 Filtrado de movimientos por fecha

```
// Obtener movimientos de una cuenta posteriores a una fecha
List<Movimiento> recientes = movimientos.stream()
    .filter(m -> m.getCuentaId().equals(cuentaId))
    .filter(m -> m.getFecha().isAfter(fechaInicio))
    .collect(Collectors.toList());
```

8.2 Filtrado por rango de fechas

```
List<Movimiento> enRango = movimientos.stream()
    .filter(m -> m.getCuentaId().equals(cuentaId))
    .filter(m -> !m.getFecha().isBefore(fechaInicio)) // fecha >= inicio
    .filter(m -> !m.getFecha().isAfter(fechaFin))    // fecha <= fin
    .sorted(Comparator.comparing(Movimiento::getFecha).reversed())
    .collect(Collectors.toList());
```

8.3 Cálculo del saldo total de un cliente

```
// Calcular el saldo total sumando los saldos de todas las cuentas del cliente
BigDecimal saldoTotal = cuentas.stream()
    .filter(c -> c.getClienteId().equals(clienteId))
    .map(Cuenta::getSaldo)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

8.4 Transformación de datos

```
// Obtener la lista de números de cuenta de un cliente
List<String> numeros = cuentas.stream()
    .filter(c -> c.getClienteId().equals(clienteId))
    .map(Cuenta::getNumeroCuenta)
    .collect(Collectors.toList());

// Ordenar cuentas por saldo descendente
List<Cuenta> ordenadas = cuentas.stream()
    .sorted(Comparator.comparing(Cuenta::getSaldo).reversed())
    .collect(Collectors.toList());
```

9. Guía de refactorización paso a paso

La migración del sistema del Módulo 1 al Módulo 2 debe hacerse de forma incremental para minimizar el tiempo en el que el proyecto está en un estado no funcional.

Paso	Descripción
1. Reorganizar paquetes	Reorganizar los paquetes siguiendo la arquitectura descrita en la Sección 2. El proyecto debe seguir compilando después de este paso.
2. Separar lógica en servicios	Extraer la lógica de negocio de los controladores o repositorios y moverla a las clases de servicio.
3. Configurar PostgreSQL	Crear la base de datos novabank y ejecutar el schema.sql del Módulo 1. Verificar que las tablas se han creado.
4. DatabaseConnectionManager	Implementar la clase con el patrón Singleton y verificar la conexión con PostgreSQL.
5. Interfaces de repositorio	Definir ClienteRepository, CuentaRepository y MovimientoRepository con todas las firmas, incluyendo las variantes con Connection.
6. Implementar repositorios JDBC	Implementar ClienteRepositoryJdbc, CuentaRepositoryJdbc y MovimientoRepositoryJdbc. Probar cada uno de forma aislada.
7. Conectar servicios con JDBC	Actualizar los servicios para que usen los repositorios JDBC. El sistema debería funcionar con la base de datos en este punto.
8. Implementar transacciones	Modificar el método transferir en OperacionService para que las tres operaciones se ejecuten en una única transacción JDBC.
9. Aplicar patrones	Implementar MovimientoFactory y ClienteBuilder (obligatorios). Implementar Strategy y/o Decorator donde aporten valor real.
10. Refactorizar con Streams	Revisar los métodos que procesan colecciones y reescribirlos con Streams donde mejore la legibilidad.
11. Actualizar tests	Mantener los tests del Módulo 1 y añadir tests de integración para los repositorios JDBC y tests unitarios de servicios con Mockito.

10. Dependencias Maven

Además de las dependencias del Módulo 1, se añaden las necesarias para la persistencia con PostgreSQL:

```
<dependencies>
    <!-- Driver JDBC para PostgreSQL -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.7.1</version>
    </dependency>

    <!-- JUnit 5 (mantenida del Módulo 1) -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.0</version>
        <scope>test</scope>
    </dependency>

    <!-- Mockito (mantenida del Módulo 1) -->
    <dependency>
        <groupId>org.mockito</groupId>
        <artifactId>mockito-core</artifactId>
        <version>5.4.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

11. Testing en el Módulo 2

La suite de tests del Módulo 1 debe mantenerse y ampliarse con pruebas específicas para la nueva arquitectura. La separación en capas facilita enormemente el testing: los servicios pueden probarse sin base de datos gracias a Mockito, y los repositorios pueden probarse de forma aislada contra una base de datos de test.

11.1 Tests de servicios con Mockito

Los servicios deben probarse de forma aislada. Mockito permite simular (mockear) los repositorios, de modo que el test verifica exclusivamente la lógica del servicio, sin depender de la base de datos:

```
@ExtendWith(MockitoExtension.class)
class OperacionServiceTest {

    @Mock
    private CuentaRepository cuentaRepository;

    @Mock
    private MovimientoRepository movimientoRepository;

    @InjectMocks
    private OperacionServiceImpl operacionService;

    @Test
    void retirar_conSaldoInsuficiente_debeLanzarExcepcion() {
        // Arrange
        Cuenta cuenta = new Cuenta(1L, "ES9121000000000000001", 1001L,
                                   "CORRIENTE", new BigDecimal("100.00"));
        when(cuentaRepository.buscarPorNumero("ES9121000000000000001"))
            .thenReturn(Optional.of(cuenta));

        // Act & Assert
        assertThrows(SaldoInsuficienteException.class, () ->
            operacionService.retirar("ES9121000000000000001",
                                     new BigDecimal("500.00"))
        );

        // Verificar que no se actualizó el saldo ni se guardó ningún movimiento
        verify(cuentaRepository, never()).actualizarSaldo(any(), any());
        verify(movimientoRepository, never()).guardar(any());
    }
}
```

11.2 Tests de integración con base de datos

Para los repositorios JDBC se recomienda implementar tests de integración que usen una base de datos de test real. Observa cómo el ClienteBuilder del apartado 8.3 simplifica la creación de objetos de prueba:

```
class ClienteRepositoryJdbcTest {

    private ClienteRepository repository = new ClienteRepositoryJdbc();

    @BeforeEach
```

```
    void limpiarDatos() {
        // Limpiar la base de datos antes de cada test
        // para garantizar independencia entre pruebas
        // p.ej.: ejecutar DELETE FROM movimientos; DELETE FROM cuentas; DELETE
FROM clientes;
    }

    @Test
    void guardar_debeAsignarIdYPersistir() {
        // Arrange: Builder hace el test mucho más legible
        Cliente cliente = new ClienteBuilder()
            .nombre("Ana")
            .apellidos("Garcia")
            .dni("11111111A")
            .email("ana@test.com")
            .telefono("600000001")
            .build();

        // Act
        Cliente guardado = repository.guardar(cliente);

        // Assert
        assertNotNull(guardado.getId());
        assertEquals("11111111A", guardado.getDni());
    }
}
```

Buena práctica

Fíjate en la sinergia entre patrones: el Builder (Sección 7.3) simplifica la creación de objetos de prueba, haciendo que los tests sean más legibles y fáciles de mantener. Este es uno de los beneficios más prácticos del patrón Builder en proyectos reales.

12. Control de versiones en el Módulo 2

En el Módulo 1 se introdujeron los fundamentos de Git: repositorio en GitHub, ramas por funcionalidad y mensajes de commit con Conventional Commits. En este módulo se da un paso adelante y se trabaja con un flujo más cercano al que el alumno encontrará en equipos de trabajo reales.

El objetivo no es añadir burocracia, sino aprender a planificar, comunicar y revisar el trabajo de forma estructurada, que es una competencia tan importante como el código en sí.

12.1 Qué cambia respecto al Módulo 1

En el Módulo 1 el alumno creó ramas siguiendo una estructura predefinida. En este módulo ese andamiaje desaparece: el alumno debe leer los objetivos del módulo, identificar las tareas que hay que realizar, decidir cómo agruparlas y planificar su propio trabajo. Esto replica el proceso real de un desarrollador al inicio de un sprint.

Las novedades que se incorporan son:

- Uso de Issues de GitHub como sistema de tickets para planificar y registrar el trabajo.
- Ramas vinculadas a issues concretos, con nombres que reflejan el ticket.
- Pull Requests (PR) como mecanismo de integración, con descripción y revisión.
- Revisión cruzada entre compañeros mediante el mecanismo de colaboradores de GitHub.

12.2 Fase de planificación: crear los Issues

Antes de escribir ninguna línea de código, el alumno debe abrir el repositorio de GitHub y crear los Issues que representan el trabajo a realizar en este módulo. Un Issue es una unidad de trabajo: puede ser una funcionalidad, una refactorización o un conjunto de tests relacionados.

> **Nota**
> ¿Qué es un Issue en GitHub?
> Un Issue es un ticket que describe una tarea, funcionalidad o problema. En equipos reales se usan para planificar sprints, registrar bugs y comunicar el estado del trabajo. GitHub los integra directamente con ramas y pull requests, formando un flujo de trabajo completo.

Pasos para crear un Issue:

4. Ve a tu repositorio en GitHub y haz clic en la pestaña Issues.
5. Pulsa New issue.
6. Escribe un título claro y conciso (p. ej.: 'Implementar capa de persistencia JDBC para clientes').
7. En la descripción, explica qué hay que hacer y qué criterios indican que está terminado (Definition of Done).
8. Asígnate el issue (campo Assignees). GitHub le asignará un número automáticamente (p. ej. #3).

El alumno debe identificar y crear sus propios issues a partir de los objetivos de este módulo. Como referencia orientativa — no como lista a seguir — algunas áreas de trabajo habituales son:

- Reorganización de paquetes y arquitectura por capas.
- Configuración de PostgreSQL e implementación del gestor de conexiones.
- Interfaces de repositorio y sus implementaciones JDBC.
- Gestión de transacciones en la transferencia.
- Patrones de diseño: Factory y Builder.
- Refactorización con Streams y Lambdas.
- Tests unitarios y de integración.

> **Buena práctica**
> Ni demasiado granular ni demasiado grande. Un issue que describa 'implementar todo el módulo 2' no ayuda a planificar. Uno por cada método de un repositorio añade ruido innecesario. El nivel adecuado es una tarea concreta que puedas completar en una o pocas sesiones de trabajo.

12.3 Estrategia de ramas

La estrategia de ramas del Módulo 1 se mantiene: main para versiones estables y develop para la integración continua. Lo nuevo es que cada rama de funcionalidad debe estar vinculada a un issue concreto.

Convención de nombrado recomendada:

```
# Formato: <tipo>/<numero-issue>-<descripcion-breve>

feature/3-persistencia-jdbc-clientes
feature/7-transacciones-transferencia
refactor/9-streams-movimientos
test/11-tests-integracion-repositorios
```

Para crear y trabajar en una rama vinculada a un issue:

```
# Partir siempre de develop actualizado
git checkout develop
git pull origin develop

# Crear y cambiar a la rama del issue #3
git checkout -b feature/3-persistencia-jdbc-clientes

# Desarrollar, hacer commits atómicos...
git add .
git commit -m "feat: implementar ClienteRepositoryJdbc con operaciones CRUD"

# Subir la rama al repositorio remoto
git push origin feature/3-persistencia-jdbc-clientes
```

> **Nota**
> También puedes crear la rama directamente desde el Issue en GitHub: en la página del issue, busca el apartado 'Development' en la columna derecha y haz clic en 'Create a branch'. GitHub crea la rama automáticamente vinculada al issue.

12.4 Pull Requests: integrar el trabajo

Cuando una tarea está terminada, en lugar de hacer merge directamente desde la terminal, se abre un Pull Request (PR) en GitHub. El PR documenta qué se ha hecho, permite revisarlo antes de fusionarlo y queda registrado en el historial del proyecto.

Cómo abrir un Pull Request:

9. Sube la rama al repositorio remoto con git push.
10. Ve a GitHub. Aparecerá un banner con 'Compare & pull request'. Pulsa el botón.
11. Selecciona develop como rama destino (base: develop).
12. Escribe un título descriptivo y una descripción que explique qué hace el PR y por qué.
13. En la descripción, referencia el issue con 'Closes #3' — GitHub cerrará el issue automáticamente al fusionar.
14. Si tienes un compañero revisor, añádelo en el campo Reviewers. Pulsa Create pull request.

> **Buena práctica**
> Una buena descripción de PR incluye: qué problema resuelve, qué decisiones técnicas se han tomado y cualquier punto sobre el que el revisor deba fijarse. Esto ahorra tiempo en la revisión y genera un historial de decisiones técnicas muy valioso a largo plazo.

Ejemplo de descripción de PR:

```
## Qué hace este PR
Implementa ClienteRepositoryJdbc con las operaciones CRUD
usando JDBC y try-with-resources para la gestión de conexiones.

## Decisiones técnicas
- Se añade variante buscarPorDni(String, Connection) para poder
  participar en transacciones gestionadas desde el servicio.
- El mapeo de ResultSet se extrae a mapearCliente() para evitar
  duplicación entre los distintos métodos de búsqueda.

## Tests
Tests de integración en ClienteRepositoryJdbcTest.
Requieren PostgreSQL activo con la base de datos novabank.

Closes #3
```

12.5 Revisión cruzada entre compañeros

Revisar el código de otra persona es una de las habilidades más valiosas en un equipo de desarrollo. Ayuda a detectar errores, compartir conocimiento y mantener la calidad. En este módulo se introduce esta práctica usando el mecanismo de colaboradores de GitHub.

Cómo añadir a un compañero como colaborador

Para que un compañero pueda revisar y aprobar tus Pull Requests en tu repositorio:

15. Ve a Settings en tu repositorio de GitHub.
16. En el menú izquierdo, selecciona Collaborators.
17. Pulsa Add people e introduce el nombre de usuario de GitHub de tu compañero.
18. Tu compañero recibirá un email de invitación y deberá aceptarla.

Cómo revisar un Pull Request

Cuando el compañero te asigne como revisor en un PR recibirás una notificación. Para revisar:

19. Abre el PR y ve a la pestaña Files changed para ver exactamente qué ha cambiado.
20. Para comentar una línea concreta, haz clic en el símbolo + que aparece al pasar el ratón por esa línea.
21. Cuando termines, pulsa Review changes (arriba a la derecha en Files changed).
22. Elige entre Comment (observaciones sin bloquear), Approve (listo para fusionar) o Request changes (hay algo que corregir antes).
23. Pulsa Submit review.

Si el revisor solicita cambios, el autor hace los ajustes con nuevos commits en la misma rama — el PR se actualiza automáticamente. Cuando el revisor aprueba, el autor fusiona el PR a develop.

> **Atención**
> La revisión de código no es una evaluación del compañero: es una colaboración. Los comentarios deben ser constructivos, técnicos y concretos.
> 'Esto está mal' no es un comentario útil. 'Aquí si la query falla antes del commit, la conexión queda abierta — considera try-with-resources' sí lo es.

12.6 Cerrar el módulo: merge a main y etiqueta de versión

Al terminar el módulo, cuando develop tiene todo el trabajo integrado y los tests pasan, se fusiona a main y se etiqueta la versión:

```
# Fusionar develop a main
git checkout main
git merge develop
git push origin main

# Etiquetar la versión del módulo
git tag -a v2.0.0 -m "Módulo 2: arquitectura por capas y persistencia JDBC"
git push origin v2.0.0
```

12.7 Resumen del flujo de trabajo

```
1. Planificar: crear Issues en GitHub con título, descripción y criterios de
finalización
   └─ Asignarse el issue · Estimar el esfuerzo

2. Crear una rama local vinculada al issue
   └─ git checkout -b feature/<num>-<descripcion>

3. Desarrollar con commits atómicos y descriptivos (Conventional Commits)
   └─ feat: · fix: · refactor: · test: · docs:

4. Subir la rama y abrir un Pull Request hacia develop
   └─ Describir el PR · Referenciar el issue con 'Closes #<num>'
   └─ Asignar revisor si aplica

5. Revisión y aprobación (propia o del compañero)
   └─ Corregir si hay cambios solicitados · Aprobar cuando esté listo
```

```
6. Fusionar el PR a develop · GitHub cierra el issue automáticamente

7. Al finalizar el módulo: merge develop → main · git tag v2.0.0
```

13. Actualización del README

El README.md debe actualizarse para reflejar la nueva arquitectura y los requisitos adicionales. Incluir como mínimo las siguientes secciones:

- Nueva arquitectura: explicación de las cuatro capas del sistema con un diagrama ASCII similar al de la Sección 2.
- Requisitos adicionales: PostgreSQL instalado y en ejecución.
- Configuración de la base de datos: cómo crear la base de datos novabank y ejecutar el schema.sql.
- Variables de conexión: cómo configurar el usuario y la contraseña de PostgreSQL en DatabaseConnectionManager.
- Ejecución del sistema: mvn clean compile y mvn exec:java.
- Ejecución de tests: mvn test (indicar que los tests de repositorios JDBC requieren PostgreSQL activo).
- Patrones de diseño aplicados: breve descripción de cada patrón y dónde se aplica en el proyecto.

13. Documento PDF entregable

El documento entregable del Módulo 2 sustituye al del Módulo 1. Debe incluir tanto lo ya documentado como los nuevos apartados específicos de este módulo:

- Arquitectura del sistema: descripción de las cuatro capas con diagrama ASCII.
- Justificación de los patrones de diseño: no basta con listarlos; el alumno debe explicar qué problema concreto resuelve cada patrón en el contexto de NovaBank, y por qué ese patrón es la solución adecuada.
- Persistencia JDBC: descripción del acceso a datos, con ejemplos de las queries más importantes y cómo se mapean los ResultSet a objetos Java.
- Transacciones: explicación de cómo se implementa la atomicidad en la transferencia, por qué se pasa la conexión como parámetro a los repositorios y qué ocurriría si no se hiciera.
- Programación funcional: ejemplos concretos del uso de Streams y Lambdas en el proyecto.
- Refactorización: comparación del código del Módulo 1 con el del Módulo 2, destacando qué ha mejorado y por qué.
- Testing: descripción de los tests implementados, diferenciando entre tests unitarios con Mockito y tests de integración con base de datos real.
- Capturas de pantalla: historial de commits de Git, ejecución exitosa de mvn test, y aplicación en funcionamiento.